

Lab 3: Solving Linear Systems & Benchmarking

CAS CS 132: Geometric Algorithms

Teaching Fellow: Angelos Poulis (apoulis@bu.edu)
Mar. 2nd 2026

Benchmarking

```
import time
start = time.time()
process(A,bs) # thing being timed
total = time.time() - start
```

For this lab, you'll benchmark linear-system solvers by:

- Time using Python's `time.time()`
- Solve many systems for the same matrix: same A , multiple bs
- Repeat across matrix sizes to see how runtime scales and compare methods

Do

- Time *only* what you mean to measure
e.g., `process(A, bs)`
- Fair comparisons: same A and bs across methods
- Sanity-check: solutions should match (up to rounding)

Don't

- Mix in unrelated things: I/O, plotting, data generation inside timed block
- Change inputs mid-benchmark: different A/bs per method
- Benchmark buggy code: if solutions differ, timing is meaningless

Three ways to solve $Ax = b$

(1) Solve

```
np.linalg.solve(A,b)
```

- Best default for one-off solves
- Re-does heavy work for each new b

(2) LU factor + solve

```
lu, piv = lu_factor(A)  
lu.solve((lu,piv),b)
```

- Factor once; reuse for many b
- Often fastest for many RHS

(3) Inverse + multiply

```
A_inv = inv(A)  
A_inv @ b
```

- Works but usually not recommended
- Can be slower / less stable than solve

Takeaway: if A is fixed and you have lots of b s, do the “one-time work” once (LU or inv) and reuse it.

Coding demo

- `rng.random()` outputs floating point numbers $\sim U(0,1)$
 - We scale to desired range `[low, hi)`
- `time.time()` returns the current time as a fp number representing the number of seconds since the Unix epoch (January 1, 1970, 00:00:00 UTC)
 - By subtracting the value of two consecutive calls (e.g., `t_solve - start`) you get the elapsed time between calls in seconds

```
import time
import numpy as np
import scipy.linalg as la

rng = np.random.default_rng(0)
low, hi = -100, 100

A = (hi - low) * rng.random((4, 4)) + low
b = (hi - low) * rng.random(4) + low

# 1) np.linalg.solve
start = time.time()
x_solve = np.linalg.solve(A, b)
t_solve = time.time() - start

# 2) LU factorization + solve
start = time.time()
lu = la.lu_factor(A)
x_lu = la.lu_solve(lu, b)
t_lu = time.time() - start

# 3) inv: x = A^-1 b
start = time.time()
Ainv = np.linalg.inv(A)
x_inv = Ainv @ b
t_inv = time.time() - start

# sanity-check: make sure we get the same result
assert np.allclose(x_solve, x_lu) and np.allclose(x_solve, x_inv)
```


- **Link to Lab 3:** <https://nmmull.github.io/cs132-s26/labs/3/main.pdf>
- **Write-up due:** Mar 19, 2026 by 8:00PM.

